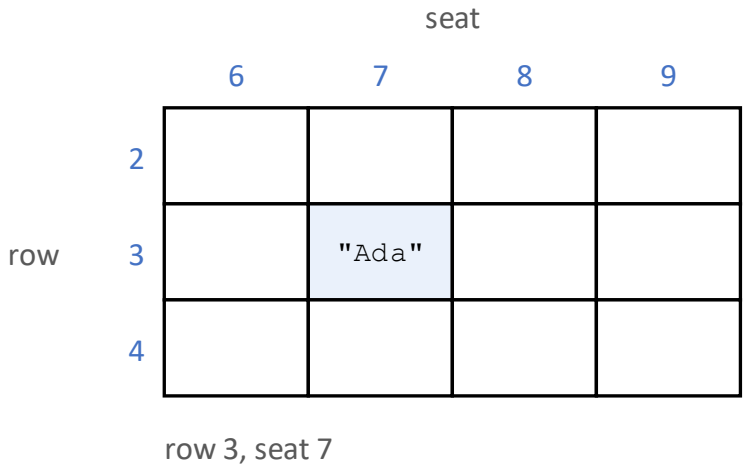
Tuples

A KEY WITH TWO PARTS

- You're making a seating chart: who sits where?
- A seat has two parts: a row and a seat number
- Last week: a list can't be a key, because it can change
- We need a sequence that **can't** change

```
seats = {}
seats[[3, 7]] = "Ada"
```

```
TypeError: unhashable type: 'list'
```

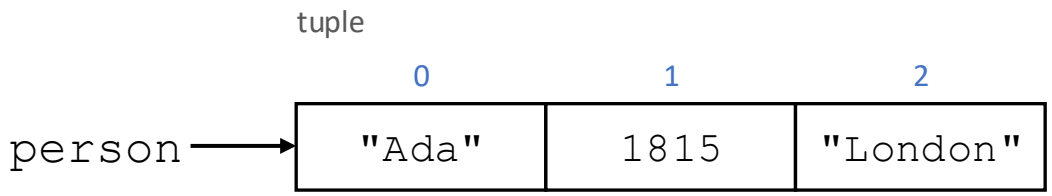


A TUPLE IS AN IMMUTABLE SEQUENCE

- A tuple is a sequence of values, like a list
- Make one with parentheses, commas between the elements
  - (parentheses are optional, but standard practice)
- Indexing, slicing, `len`, `in`, and `for` all work the same way they do on lists
- Unlike lists, tuples are *immutable*.

```
person = ("Ada", 1815, "London")
print(person[0])
print(person[1:])
print(len(person))
print(1815 in person)
```

```
Ada
(1815, 'London')
3
True
```



MAKING TUPLES

- Python makes a tuple out of values with commas between them; the parentheses are optional
- A one-element tuple needs a comma at the end
- Parentheses around one value don't make a tuple
- Standard practice is to write the parentheses when *making* a tuple, but not when unpacking or returning
- `tuple(s)` makes a tuple from any sequence; `()` is the empty tuple

```
t = "Ada", 1815
one = ("Ada",)
two = ("Ada")
print(type(t))
print(type(one))
print(type(two))
print(tuple("abc"))
```

Predict: what type is `two`?

Predict: what does the final line print?

MAKING TUPLES

- Python makes a tuple out of values with commas between them; the parentheses are optional
- A one-element tuple needs a comma at the end
- Parentheses around one value don't make a tuple
- Standard practice is to write the parentheses when *making* a tuple, but not when unpacking or returning
- `tuple(s)` makes a tuple from any sequence; `()` is the empty tuple

```
t = "Ada", 1815
one = ("Ada",)
two = ("Ada")
print(type(t))
print(type(one))
print(type(two))
print(tuple("abc"))
```

```
<class 'tuple'>
<class 'tuple'>
<class 'str'>
('a', 'b', 'c')
```

SAME MOVE, DIFFERENT RESULT

- Tuples are immutable, like strings: you can't change an element
- There's no `append` or `remove` either: a tuple has no methods that change it

list

```
names = ["Ada", "Grace"]
names[0] = "Alan"
print(names)
```

```
['Alan', 'Grace']
```

tuple

```
names = ("Ada", "Grace")
names[0] = "Alan"
```

```
TypeError: 'tuple' object does not
support item assignment
```

ALIASING A TUPLE IS SAFE

- "Changing" a tuple means building a new one
- `+` and `*` build new tuples, just like with strings
- The old tuple is still there for anyone else who refers to it
- So a tuple, like a string, is safe to share

```
a = ("Ada", 1815)
b = a
a = a + ("London",)
print(a)
print(b)
```

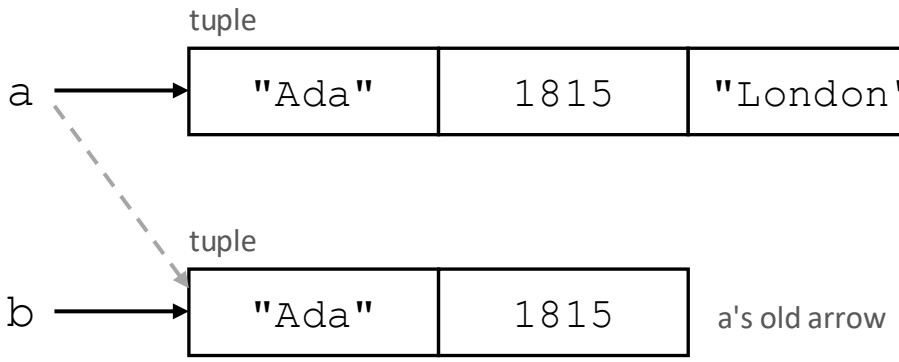
Predict: what do the last two lines print?

ALIASING A TUPLE IS SAFE

- "Changing" a tuple means building a new one
- `+` and `*` build new tuples, just like with strings
- The old tuple is still there for anyone else who refers to it
- So a tuple, like a string, is safe to share

```
a = ("Ada", 1815)
b = a
a = a + ("London",)
print(a)
print(b)
```

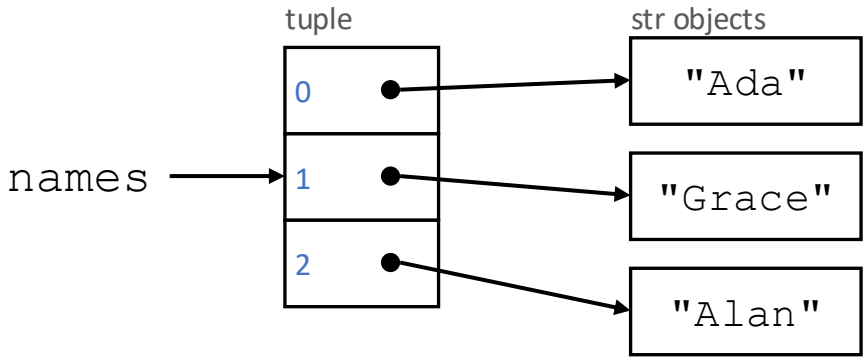
```
('Ada', 1815, 'London')
('Ada', 1815)
```



WHAT A TUPLE LOOKS LIKE INSIDE

- Like a list, a tuple holds references (arrows) to its elements
- Once a tuple is made, its arrows are fixed
- No arrow can be pointed somewhere else, and no arrows can be added or taken away

```
names = ("Ada", "Grace", "Alan")
```



FIXED ARROWS, NOT FIXED OBJECTS

- A tuple can't change which objects it refers to
- But an object it refers to can still change, if it's mutable
- Here the list inside the tuple gets a new element
- The tuple's arrows never moved

```
team = ("Ada", ["Grace", "Alan"])
team[1].append("Katherine")
print(team)
```

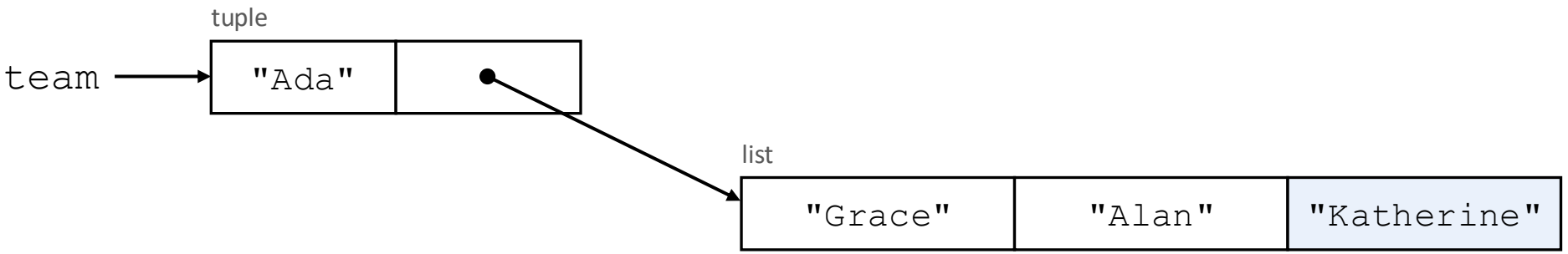
Predict: does line 2 work? If it does, what does line 3 print?

FIXED ARROWS, NOT FIXED OBJECTS

- A tuple can't change which objects it refers to
- But an object it refers to can still change, if it's mutable
- Here the list inside the tuple gets a new element
- The tuple's arrows never moved

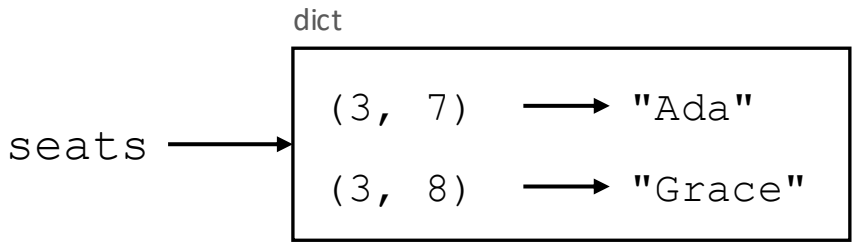
```
team = ("Ada", ["Grace", "Alan"])
team[1].append("Katherine")
print(team)
```

```
('Ada', ['Grace', 'Alan', 'Katherine'])
```



TUPLES AS KEYS

- A tuple can't change, so it can be a dictionary key
- No extra parentheses needed: `seats[3, 7]` works
- Back to the seating chart: one key per seat, no list needed
- Tuples can be values, too




```
seats = {}
seats[3, 7] = "Ada"
seats[(3, 8)] = "Grace"
print(seats[3, 7])
print((3, 8) in seats)
print(seats)
```

```
Ada
True
{(3, 7): 'Ada', (3, 8): 'Grace'}
```

QUESTIONS YOU MIGHT BE ASKING

- Why not use tuples for everything? **You can't change them.** A list that grows needs to be a list
- So when do I use a tuple? **When the parts belong together** and there's a fixed number of them: row and seat, name and year
- If a group of values shouldn't change, a tuple keeps it from changing by accident
- Can any tuple *always* be a key? **No:** not if there's a list inside it

```
team = ("Ada", ["Grace", "Alan"])
d = {}
d[team] = "red"
```

```
TypeError: unhashable type: 'list'
```

TUPLE ASSIGNMENT

- Put a tuple of variables on the left of =
- Each one gets the value in the same position on the right
- The right side can be any sequence, including the list that `split` returns

```
name, year = "Ada", 1815
print(name)
print(year)

user, domain = "ada@python.org".split("@")
print(user)
print(domain)
```

```
Ada
1815
ada
python.org
```

TANGENT!: SWAP TWO VARIABLES

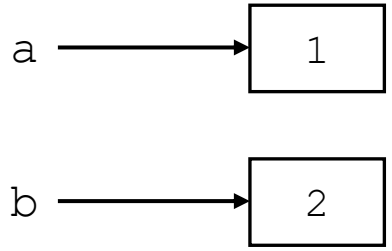
- `a` refers to `1` and `b` refers to `2`
- Write code that swaps them: afterward `a` is `2` and `b` is `1`
- Work it out on paper first

```
a = 1
b = 2
# your code here
print(a, b)
```

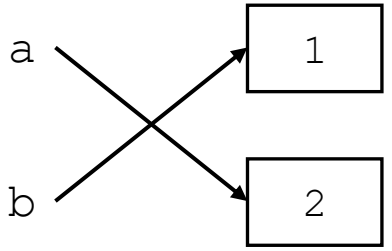
should print:

```
2 1
```

before



goal



TANGENT!: SWAPPING: ONE SOLUTION

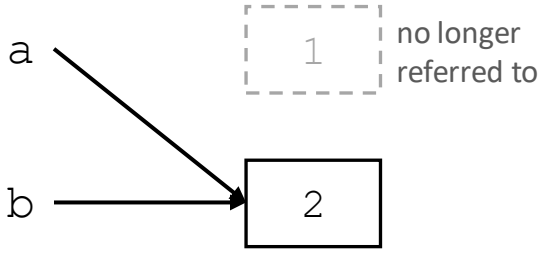
- The first try loses a value: after `a = b`, nothing refers to `1`
- Fix: save one value in a third variable before you overwrite it

first try

```
a = 1
b = 2
a = b
b = a
print(a, b)
```

```
2 2
```

after a = b

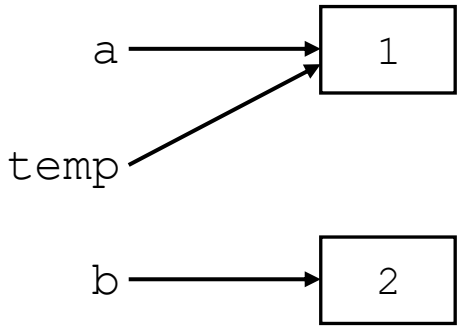


with a temporary variable

```
a = 1
b = 2
temp = a
a = b
b = temp
print(a, b)
```

```
2 1
```

after temp = a

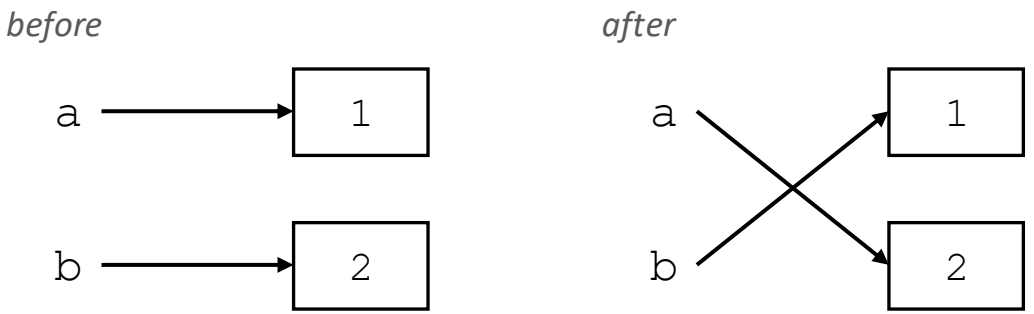


SWAPPING WITH TUPLE ASSIGNMENT

- Python works out the whole right side first, then assigns
- So `a, b = b, a` swaps two variables in one line
- No temporary variable needed

```
a = 1
b = 2
a, b = b, a
print(a, b)
```

```
2 1
```



A BUG YOU'LL WRITE THIS WEEK

- The number of variables on the left has to match the number of values on the right
- Too many values, or too few, is a `ValueError`
- The message says how many it expected
- When you see it, print the value and check its shape

```
name, year = ("Ada", 1815, "London")
```

```
ValueError: too many values to unpack (expected 2)
```

LOOPING WITH ITEMS

- `items()` gives a dictionary's items as (key, value) tuples
- Tuple assignment in the `for` line unpacks each one
- Keys and values together, no lookup inside the loop

```
rooms = {"Ada": 137, "Grace": 204}
print(list(rooms.items()))

for name, room in rooms.items():
    print(name, room)
```

```
[('Ada', 137), ('Grace', 204)]
Ada 137
Grace 204
```

RETURNING MORE THAN ONE VALUE

- A function can only return one value…
- …but that value can be a tuple
- Unpack it with tuple assignment where you call the function
- `divmod(a, b)` returns `a // b` and `a % b` together

```
q, r = divmod(7, 3)
print(q, r)

def min_max(t):
    return min(t), max(t)

low, high = min_max([86, 94, 75])
print(low, high)
print(min_max([86, 94, 75]))
```

```
2 1
75 94
(75, 94)
```

PACKING ARGUMENTS

- A * in front of a parameter packs all the arguments into one tuple
- So a function can take any number of arguments
- A * in front of an argument does the reverse: it unpacks a sequence into separate arguments

```
def mean(*args):
    return sum(args) / len(args)

print(mean(1, 2, 3))

scores = [86, 94, 75]
print(mean(*scores))
```

```
2.0
85.0
```

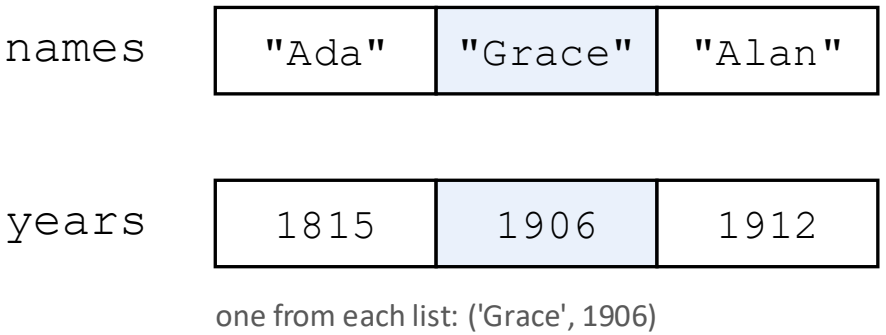
ZIP PAIRS THINGS UP

- `zip` takes two sequences and pairs up their elements, one from each
- It gives back a zip object, which hands out tuples when you loop over it
- Wrap it in `list` to see them all

```
names = ["Ada", "Grace", "Alan"]
years = [1815, 1906, 1912]
print(zip(names, years))
print(list(zip(names, years)))
```

```
<zip object at 0x7f3a2c1e5b40>
[('Ada', 1815), ('Grace', 1906), ('Alan', 1912)]
```

(your address will differ)



ENUMERATE

- `enumerate` hands out (index, element) tuples
- Use it when you need the position and the element
- Unpack them in the `for` line, as with `zip`

```
names = ["Ada", "Grace", "Alan"]
for i, name in enumerate(names):
    print(i, name)
```

```
0 Ada
1 Grace
2 Alan
```

WHEN THE LENGTHS DON'T MATCH

- `zip` works on any sequences, even a string and a list
- What if one is shorter than the other?

```
letters = "abcd"
numbers = [1, 2, 3]
print(list(zip(letters, numbers)))
```

Predict: what does the last line print?

WHEN THE LENGTHS DON'T MATCH

- `zip` works on any sequences, even a string and a list
- What if one is shorter than the other?
- `zip` **stops as soon as the shorter one runs out**
- The extra `"d"` is ignored, with no error

```
letters = "abcd"
numbers = [1, 2, 3]
print(list(zip(letters, numbers)))
```

```
[('a', 1), ('b', 2), ('c', 3)]
```

LOOPING OVER TWO LISTS

- Loop over the zip object and unpack each pair
- Each time through, you get one element from each list
- Here: count the games team 1 won

```
scores1 = [1, 2, 4, 5, 1, 5, 2]
scores2 = [5, 5, 2, 2, 5, 2, 3]

wins = 0
for team1, team2 in zip(scores1, scores2):
    if team1 > team2:
        wins += 1
print(wins)
```

```
3
```

Making a dictionary from two lists

- Last week: building a dictionary from two lists took a loop and `range(len(...))`
- `zip` makes the pairs; `dict` turns a sequence of pairs into a dictionary

last week

```
born = {}
for i in range(len(names)):
    born[names[i]] = years[i]
print(born)
```

```
{'Ada': 1815, 'Grace': 1906, 'Alan': 1912}
```

today

```
born = dict(zip(names, years))
print(born)
```

```
{'Ada': 1815, 'Grace': 1906, 'Alan': 1912}
```

REVIEW: CALLS INSIDE CALLS

- The result of one call can be the argument to another
- Python works from the inside out: the innermost call runs first
- Methods can be chained: each one is called on what the one before it returned

```
born = dict(zip(names, years))

email = "  Ada@Python.org "
print(email.strip().lower().split("@"))
```

Predict: what does the last line print? What does each step hand to the next?

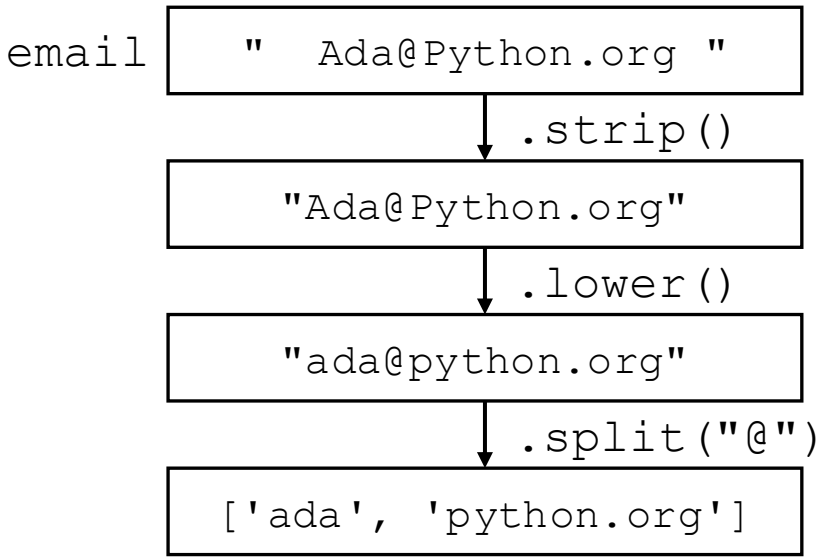
REVIEW: CALLS INSIDE CALLS

- The result of one call can be the argument to another
- Python works from the inside out: the innermost call runs first
- Methods can be chained: each one is called on what the one before it returned

```
born = dict(zip(names, years))

email = "  Ada@Python.org "
print(email.strip().lower().split("@"))
```

```
['ada', 'python.org']
```



COMPARING TUPLES

- Python compares tuples one element at a time, left to right
- The first elements that differ decide the answer
- Everything after that is never looked at

```
print((0, 1, 2) < (0, 3, 4))
print((0, 1, 2000000) < (0, 3, 4))
print(("Grace", 1) < ("Ada", 99))
```

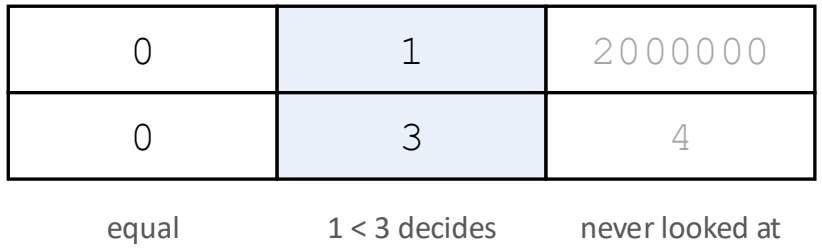
Predict: what does each line print? Watch line 2 closely

COMPARING TUPLES

- Python compares tuples one element at a time, left to right
- The first elements that differ decide the answer
- Everything after that is never looked at

```
print((0, 1, 2) < (0, 3, 4))
print((0, 1, 2000000) < (0, 3, 4))
print(("Grace", 1) < ("Ada", 99))
```

```
True
True
False
```



SORTING A LIST OF TUPLES

- `sorted` works on a sequence of tuples, too
- It compares them the way the last slide did: first elements first
- For dictionary items, that means sorting by key

```
counts = {"b": 1, "a": 3, "n": 2}
print(sorted(counts.items()))
```

Predict: what order do the items come out in?

SORTING A LIST OF TUPLES

- `sorted` works on a sequence of tuples, too
- It compares them the way the last slide did: first elements first
- For dictionary items, that means sorting by key

```
counts = {"b": 1, "a": 3, "n": 2}
print(sorted(counts.items()))
```

```
[('a', 3), ('b', 1), ('n', 2)]
```

A FUNCTION IS A VALUE

- def makes a function object and gives it a name, the way = does for other values
- The name without parentheses is the function itself
- With parentheses, it's a call: the function runs and you get back what it returns

```
def shout(s):
    return s.upper() + "!"

print(shout("hi"))
print(shout)

yell = shout
print(yell("hi"))
```

Predict: what does each print line print?

A FUNCTION IS A VALUE

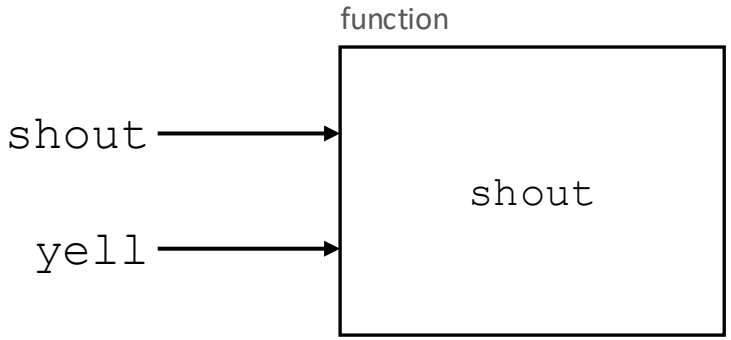
- def makes a function object and gives it a name, the way = does for other values
- The name without parentheses is the function itself
- With parentheses, it's a call: the function runs and you get back what it returns

```
def shout(s):
    return s.upper() + "!"

print(shout("hi"))
print(shout)

yell = shout
print(yell("hi"))
```

```
HI!
<function shout at 0x7f3a2c1e5b40>
HI!
(your address will differ)
```



`shout` and `yell` are aliases for the same function object

PASSING A FUNCTION INTO ANOTHER FUNCTION

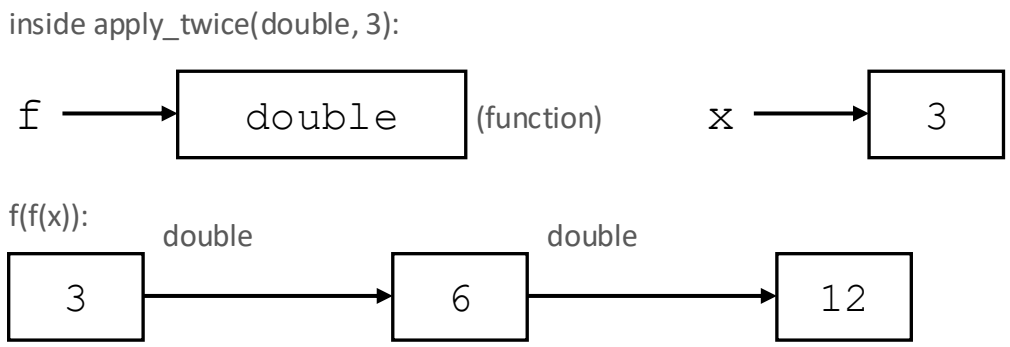
- Since a function is a value, you can pass it as an argument
- The function you pass in gets a parameter name; the receiving function calls it by that name
- Here f is whatever function was passed in

```
def apply_twice(f, x):
    return f(f(x))

def double(n):
    return n * 2

print(apply_twice(double, 3))
print(apply_twice(shout, "hi"))
```

```
12
HI!!
```



NAMES ARE JUST ARROWS

- = points a name at a value
- A function's name is a name like any other
- So what happens if you point it somewhere else?

```
def greet(name):
    return "Hi, " + name

hello = greet
greet = len
print(hello("Ada"))
print(greet("Ada"))
```

Predict: what do the last two lines print?

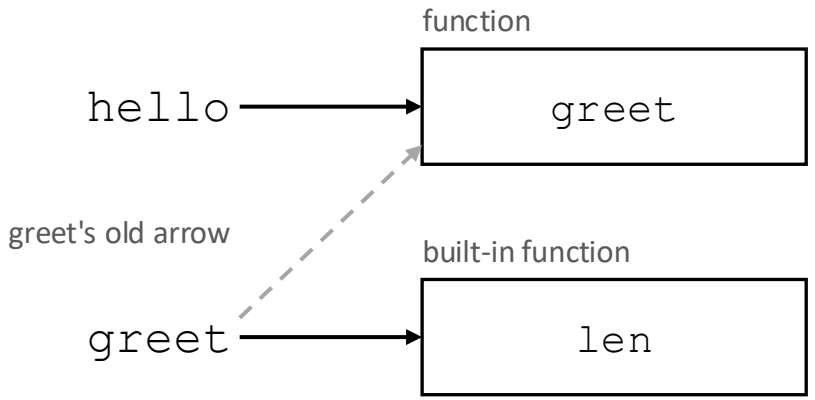
NAMES ARE JUST ARROWS

- `=` points a name at a value
- `greet = len` moves greet's arrow; the function it pointed at doesn't change
- `hello` still points at that function
- Same as aliasing a tuple: moving one arrow leaves the others alone

```
def greet(name):
    return "Hi, " + name

hello = greet
greet = len
print(hello("Ada"))
print(greet("Ada"))
```

```
Hi, Ada
3
```

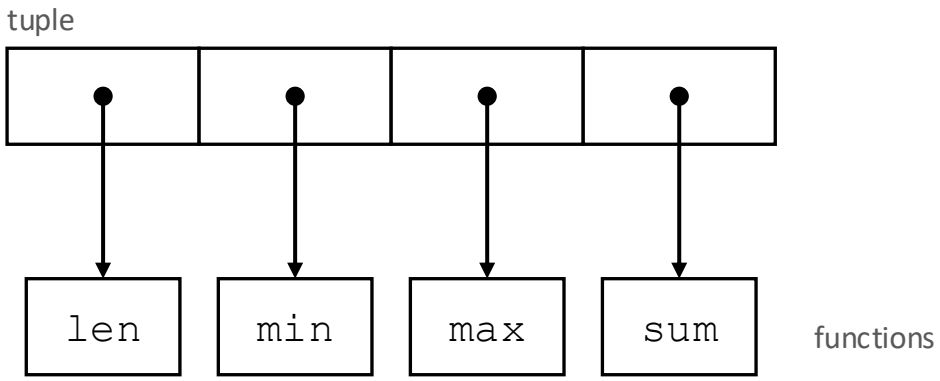


A TUPLE OF FUNCTIONS

- Built-in functions are values too: `len`, `min`, `max`, `sum`
- So a tuple can hold them, like any other value
- Each time through the loop, `f` refers to a different function

```
scores = [86, 94, 75]
for f in (len, min, max, sum):
    print(f(scores))
```

```
3
75
94
255
```



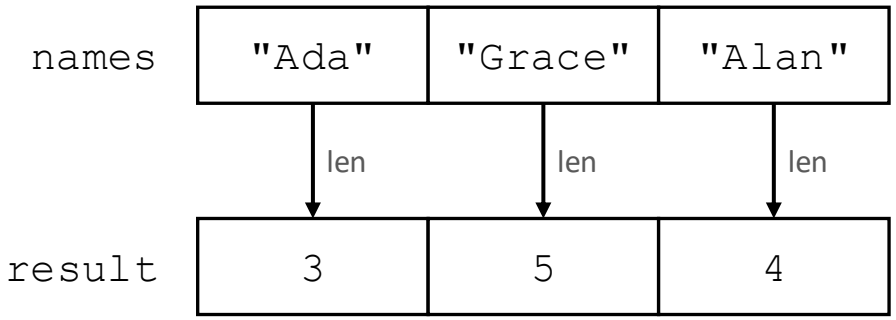
WRITE THE LOOP ONCE

- The loop is the same no matter what you want done to each item
- The caller passes in the what: any function that takes one argument
- Python has this built in: `map`

```
def apply_to_each(f, items):
    results = []
    for item in items:
        results.append(f(item))
    return results

names = ["Ada", "Grace", "Alan"]
print(apply_to_each(len, names))
print(apply_to_each(shout, names))
```

```
[3, 5, 4]
['ADA!', 'GRACE!', 'ALAN!']
```



SORTING BY SOMETHING ELSE

- `key=` tells `sorted` what to sort by
- Give it a function: `sorted` calls it on each element and sorts by what it returns
- `max` and `min` take a `key` too

```
names = ["Grace", "Ada", "Barbara", "Alan"]
print(sorted(names))
print(sorted(names, key=len))
print(max(names, key=len))
```

Predict: what do lines 3 and 4 print?

SORTING BY SOMETHING ELSE

- `key=` tells `sorted` what to sort by
- Give it a function: `sorted` calls it on each element and sorts by what it returns
- `max` and `min` take a `key` too

```
names = ["Grace", "Ada", "Barbara", "Alan"]
print(sorted(names))
print(sorted(names, key=len))
print(max(names, key=len))
```

```
['Ada', 'Alan', 'Barbara', 'Grace']
['Ada', 'Alan', 'Grace', 'Barbara']
Barbara
```

item		len(item)
`"Ada"`	→	3
`"Alan"`	→	4
`"Grace"`	→	5
`"Barbara"`	→	7

sorted puts the names in the order of these numbers

BACK TO THE COUNTS

- Back to the question from a few slides ago: how do we sort by count?
- Each item is a (letter, count) tuple, so the count is `t[1]`
- Write a function that returns it, and hand that function to `key=`

```
def second_element(t):
    return t[1]

counts = {"b": 1, "a": 3, "n": 2}
print(sorted(counts.items(), key=second_element))
print(max(counts.items(), key=second_element))
```

```
[('b', 1), ('n', 2), ('a', 3)]
('a', 3)
```

item		second_element(item)
`('b', 1)`	→	`1`
`('a', 3)`	→	`3`
`('n', 2)`	→	`2`

NO PARENTHESES

- `key=second_element` hands `sorted` the function itself
- `key=second_element()` calls it right now, with no arguments
- `sorted` does the calling, once for each element

```
counts = {"b": 1, "a": 3, "n": 2}
sorted(counts.items(), key=second_element())
```

```
TypeError: second_element() missing 1
required positional argument: 't'
```

PREDICT THE OUTPUT

- Work these out on paper before running them
- Then check your answers in Colab
- Draw the arrows!
- Watch the last one closely

```
f = max
print(f([3, 9, 4]))
f = min
print(f([3, 9, 4]))

print(apply_twice(shout, "ok"))

words = ["pear", "fig", "apple"]
print(max(words, key=len))
print(sorted(words, key=len))

print(sorted(words, key=len()))
```

PREDICT THE OUTPUT: ANSWERS

- **1:** `f` is whichever function it points at right now: first `max`, then `min`
- **2:** `shout("ok")` is `"OK!"`, then `shout("OK!")` is `"OK!!"`
- **3:** `key=len` compares lengths, not alphabetical order
- **4:** `len()` runs right away, with nothing to measure, before `sorted` even starts

```
f = max
print(f([3, 9, 4]))                    → 9
f = min
print(f([3, 9, 4]))                    → 3

print(apply_twice(shout, "ok"))        → OK!!

words = ["pear", "fig", "apple"]
print(max(words, key=len))             → apple
print(sorted(words, key=len))          → ['fig', 'pear', 'apple']

print(sorted(words, key=len()))        → TypeError
```

```
len() takes exactly one argument
(0 given)
```